

FINDING

BUG-MARKDOWN-EMPTY-PAYLOAD — Root-Cause Finding

Revision: 1 **Last modified:** 2026-06-16T15:26:40Z **Investigation:** READ-ONLY systematic-debugging (superpowers Iron Law + §11.4.6 + §11.4.102) **Status:** ROOT CAUSE PROVEN (FACT, wire-level reproduction, no API key)

1. FACT root cause

The defect is an **LLM-client parameter-contract mismatch at the send site**, NOT a block-splitter, whitespace-filter, wrong-field, or structure-mismatch defect, and NOT a Step-1 (EPUB→MD) defect.

The contract (FACT)

pkg/translator/llm/openai.go:152–159 builds the chat request with the user message content taken **only from the 2nd parameter (prompt)** and **ignores the 1st parameter (text)**:

```
// openai.go:121 func (c *OpenAIClient) Translate(ctx, text
    string, prompt string)
request := OpenAIRequest{
    Model: model,
    Messages: []Message{
        {Role: "user", Content:
            prompt}, // openai.go:155 – uses prompt, ignores text
    },
    ...
}
```

DeepSeekClient embeds *OpenAIClient (pkg/translator/llm/deepseek.go:9–11) and inherits this Translate verbatim. The bridge's BestClient returns a markedClient{*OpenAIClient} (pkg/bridge/bridge.go:339–346, 413–419) that delegates Translate verbatim (no arg swap). So **content must be passed in the 2nd arg**. The bridge's own internal path already knows this — see the explicit comment + correct call at pkg/bridge/bridge.go:367–369:

```
// OpenAIClient.Translate sends `prompt` (2nd arg) as the single
// user message
// and ignores `text` (1st arg); pass the composed prompt there.
return client.Translate(ctx, "", full)
```

The bug site (FACT — file:line)

cmd/markdown-translator/main.go:243-245 — Step 3's translate callback calls the client the WRONG way: real block content in the 1st arg, **empty string in the 2nd arg**:

```
mdTranslator := markdown.NewMarkdownTranslator(func(text string)
    (string, error) {
        return workflowCfg.LLMProvider.Translate(ctx, text, "") //
        <-- prompt = "" → empty user message
    })
```

workflowCfg.LLMProvider is bridge.BestClient(...) (main.go:312, bridgeWorkflowConfig), i.e. the raw OpenAI-compatible client. With prompt="" the API receives a **user message whose content is empty**. DeepSeek replies with its real "It seems you may have accidentally sent an empty message..." boilerplate, which MarkdownTranslator writes into the output AS the translation → real chapter text lost, output garbage. The markdown markers (#, ---, frontmatter) are preserved because MarkdownTranslator only routes the *inner text* to the LLM — hence the wave7 output kept structure but replaced every translatable line's text with boilerplate.

Classification of the three hypotheses in the dispatch

- block-splitter yielding empty segments → **NO** (proven below: 0 empty segments sent).
- wrong field read (header/whitespace instead of body) → **NO** (the markdown layer extracts the correct body text and passes it as text).
- structure mismatch → **NO**.
- **Actual: caller[?]client argument-contract mismatch — content placed in the ignored text arg; the consumed prompt arg left empty.**

2. Reproduction (deterministic, READ-ONLY, no API key)

Two temporary probe tests were written, run, and then DELETED (never staged/committed).

Probe A — markdown layer sends NO empties (rules out split/whitespace/Step-1)

Fed the verbatim wave7 source MD (and, separately, a real EPUB through the real EPUBToMarkdownConverter.ConvertEPUBToMarkdown) into the real MarkdownTranslator with a recording stub. Result:

```

=== STEP 1 OUTPUT (converter sourceMD), byte-quoted ===
"---\ntitle: Wave7 Test Book\nlanguage: en\n---\n\n# Wave7 Test
Book\n\n---\n\n# The Lighthouse\n\n\nThe old man walked along
the shore at dawn...\n\n\nHe remembered his youth and smiled
quietly.\n\n---\n\n"
=== PAYLOADS SENT TO LLM (4) ===
payload[0] = "Wave7 Test Book"
payload[1] = "The Lighthouse"
payload[2] = "The old man walked along the shore at dawn. The sea
was calm and the sky turned orange."
payload[3] = "He remembered his youth and smiled quietly."
=== EMPTY PAYLOADS: 0 ===

```

Step-1 converter output is byte-identical to the committed evidence
 BUG_markdown_translator_source_was_correct.md. The markdown layer
 extracts the correct text and sends it as the 1st arg with NO empties. The garbage is
 NOT produced when the stub reads the 1st arg (as MockLLMClient does) — which
 is why existing mock-based tests never caught it.

Probe B — wire-level proof against a DeepSeek simulator (the SMOKING GUN)

A real DeepSeekClient (via embedded OpenAIClient) pointed at an httptest
 server that simulates DeepSeek (reads the user message; returns "empty message"
 boilerplate iff the user content is empty, else echoes):

```

(A) Translate(text="The Lighthouse", prompt="") -> "It seems you
may have accidentally sent an empty message. ..."
(B) Translate(text="", prompt="The Lighthouse") ->
"ES::The Lighthouse"
>>> REPRODUCED at wire: content-in-text/empty-prompt -> empty user
message -> boilerplate

```

Call (A) is exactly cmd/markdown-translator/main.go:244. It produces the
 wave7 boilerplate. Call (B) (the bridge.go:369 convention) translates correctly. This
 proves the root cause at the wire, with no live API call.

3. Precise fix recommendation

Primary fix (the bug site). In cmd/markdown-translator/main.go:243–245,
 pass the block content in the **2nd arg** (the consumed prompt), matching the
 bridge.go:369 convention:

```

mdTranslator := markdown.NewMarkdownTranslator(func(text string)
    (string, error) {
        // OpenAI/DeepSeek client (bridge.BestClient) uses the 2nd arg
        as the user
        // message and ignores the 1st. Build a real translation
        instruction that
        // embeds the block text, and pass it as `prompt`.

```

```

    prompt := buildTranslationPrompt(text, *targetLang) // e.g.
        "Translate to <lang>...\n\n<text>\n\nTranslation:"
    return workflowCfg.LLMProvider.Translate(ctx, text, prompt)
})

```

Mirror the existing, working prompt builder in pkg/markdown/simple_workflow.go:82–92 (which already composes a real prompt containing the text and passes Translate(ctx, text, prompt) — that path is NOT broken precisely because its 2nd arg is non-empty). The cmd binary should use the same pattern instead of "".

Guard at the client boundary (defence-in-depth, recommended). The deeper smell is the silent ignore of the text arg and the absence of an empty-user-message guard. Recommend ONE of:

- In OpenAIClient.Translate, when prompt == "" fall back to using text as the user content (so the documented (text, context) convention some callers use — e.g. pkg/api/batch_handlers.go:146, pkg/api/handler.go:833 — also works); AND
- Refuse to send an empty/whitespace-only user message: return an explicit error rather than letting the model emit boilerplate that gets stored as a translation (§11.4.69 — never write provider boilerplate as if it were real output).

Note the **systemic contract inconsistency** (worth a follow-up item, do NOT fix blindly here): call sites disagree on which arg carries content. Content-in-1st-arg: batch_handlers.go:146, handler.go:833, coordinator.go:308, coordination/multi_llm.go:445/529, cmd/markdown-translator/main.go:244. Content-in-2nd-arg: bridge.go:369, preparation/coordinator.go:290/361/424, verification/notes.go:166, verification/polisher.go:563. Several of the content-in-1st-arg sites (those reaching a bare OpenAI/DeepSeek client with an empty 2nd arg) carry the SAME latent bug — they should be audited under §11.4.118 discovery-pressure. The empty-user-message guard above would convert any such latent occurrence from silent boilerplate-garbage into a loud error.

4. RED-test shape (§11.4.115 reproduce-first + §11.4.135 standing guard)

Two complementary RED guards, polarity-switchable (RED_MODE=1 reproduces on the pre-fix artifact, RED_MODE=0 is the standing GREEN regression guard):

1. **Wire-level client-contract guard** (pkg/translator/llm): real DeepSeekClient (embedded OpenAIClient) against an httptest DeepSeek simulator that returns "empty message" boilerplate iff the received user message is empty. Assert that calling Translate with the real block content produces a real (non-boilerplate) translation and that NO empty user message is ever sent. RED on pre-fix: Translate(ctx, "The Lighthouse", "") yields the boilerplate.
2. **End-to-end EPUB→MD→translate round-trip guard** (cmd/markdown-translator or pkg/markdown): build a real minimal EPUB ("# The Lighthouse" + 2 paragraphs), run the actual Step-1 converter, then drive Step-3

through a client wired exactly as the cmd binary wires it (bridge.BestClient-shaped, OpenAI/DeepSeek-compatible user-message-from-2nd-arg semantics) against the httptest simulator. Assert the translated MD/EPUB **contains the translated chapter text** and **contains NO** "accidentally sent an empty message" boilerplate and **no empty payload** was sent. This guards the real caller[?] client seam, not the MockLLMClient (which reads the 1st arg and masks the bug).

Both must FAIL on the current pre-fix cmd/markdown-translator/main.go:244 and PASS after the fix. Register guard #2 into the standing regression suite per §11.4.135 / §11.4.146 STEP 3 (extend across header + paragraph + list + blockquote blocks).

5. §11.4.6 honesty boundary

- PROVEN as FACT (wire reproduction): the empty-prompt/ignored-text contract mismatch at cmd/markdown-translator/main.go:244 is the root cause of the wave7 empty-payload garbage.
- UNCONFIRMED: that every other content-in-1st-arg call site is currently broken — some wrap the raw client in LLMTranslator/other layers that may compose a prompt. They share the latent pattern and MUST be audited (§11.4.118), but each needs its own per-site confirmation before being called broken.
- No files were committed or staged by this investigation; both probe tests were deleted after capturing the evidence above.